

Project 2

Katherie Botticelli

This work complies with the JMU Honor Code.

The code process:

Before this code is run a bash script must be run to create the users certificates and the server certificate this requires them to have the same ca.key and ca.crt files to allow for them to be signed by the same key and certificates. The CA gives them public keys that allow for the other certificates that are signed by the certificate authority to be able to verify their identity using their public keys. The Client and server private keys are not swapped but they check each other using their unique public keys. For the clients in the real world this means they all have their own certificates for this code. My clients are currently sharing one.

This code is a cloud server that allows clients to upload and download files from its system. It uses SSL and Certificate handshake to establish an encrypted connection to HTTPS connection between the clients and the server allowing for mutual authentication during the handshake. Meaning the client authenticates the server and the server authenticates the client. The server then authenticates the user a second time after connection is established by asking the user for a Login. If the user does not have a username or password on the server they can create an account that asks them for a username(this must be unique) a password, to verify that password then for 3 security questions that can be used for when a user forgets their password and needs to recover it. The client enters the password, verifying password, and security questions into prompts that show the (*****) as their input but saves their input as plain text not asterisks. The user is then asked after they create an account if they would like to (L) Login

(C) Create Account

(E) Exit

The user's information is saved with the salt for the password and only the hashed password is saved, not the plaintext password. This is for the safety of clients' information so that no one can see the plaintext of their passwords. The salt is saved because it is randomly generated. It needs to be saved for verification of the password or else the server would never be able to authenticate the user's password on log in. The same logic is applied to the security questions they are entered by the user in plain text and then all three answers are put together into one long string, hashed and then saved with the salt to allow for verification. The security questions are for when a user needs to change the password to allow for a form of user identification before being allowed to reset the passwords.

The user answers the prompt, lets pretend they said (L) so they are logging in. This asks them for their username and checks the given input against the saved usernames in the ([self.db](#), a dictionary that saves the users as instances that look like {'username' : self.username{ 'passwd': password , 'salt' : passwords salt, 'secureQ': hashed security questions, 'SecureSalt': salt for the security questions, 'lockedBit': integer 0 or 1, 'dir':the users directory path on the server}} . This is to keep track of the users and their data as it changes and updates throughout their time on the server. It always then login to the account and that is when the server triggers for it to go into the Katherine(Server) directory and go into the users specific directory that stores their files and only theirs. They can then download, upload and logout from here.

(U) Upload

(D)Download

(A)Account settings - which has been disabled for this Project 2

(*)Logout

The function that checks the users output uses the authBit and authUser bit checking that both are set to 1 to make sure that the user accessing these options is a logged in user as only the logged in user is able to access these specific options. The Logout works by keeping the thread alive but removing user specific data such as directory and username. Then it changes the authBit and authUser back to 0 and sets the self.username to “Guest User” this is revoking all the users privileges in the server but not disconnecting it from the server. They are then prompted back to the original prompts and options.

(L) Login

(C) Create Account

(E) Exit

The self.authBit and self.authUser bit is used to make sure that only those in specific parts of the code such as logging in but forgot password or logged in fully or not even logged in users, can not do unwanted actions in the server such as download a file or Upload on or change the password of a user. These are to make sure that the user input can not be used against the server to undermine the privileges system given to the users and keep actions separate in the server.

The Libraries:

This code is a Server and client cloud that allows clients to upload and download files off a server. For this part of my code I used.

- Thread
- ssl
- re

- maskpass
- bcrypt
- openssl

These libraries help me expand my code into a more secure cloud system.

ssl: Is a library that creates a ssl secure connection between a client and a server by using certificates that are signed by the same authentication client the server has a certificate and the client has a certificate. After they connect they initiate a handshake where they complete mutual authentication by training their certificates and checking them against the public keys given to them by the certificate authentication client. In this case my ca.key and ca.crt files. To create these files I used Openssl that is in a C code for the user to execute on each other and client.

These are for the creation of the certificate. Using openssl by executing commands in the linux terminal that create a key for my certificate authority, then I make a certificate for my certificate authority called ca.crt. Then I create the server key and the servers crt and then sign it using the ca.key and ca.crt this makes sure to establish the connection between client and server by also naming the sever.pem. I do the same steps in client with the ca.key and ca.crt after creating the client.key and clientcert to sign the client certificate and name it client.pem. These are exchanged and checked in the handshake process that creates a secure connection between my clients and allows the socket between the server and client to be encrypted. This is always for security between my communications.

Threading: Threading is a way for a computer to seem like it is completing multiple tasks at once. I did this to allow my server to accept more than one client. It works by my server accepting a client and then that client triggering a separate thread to be created as well as an instance of that thread being added to the array object of threads that I have. This allows for my

client to hypothetically run what seems like simultaneous action on the computer but is actually just the processor of my server stopping and starting the processes as it seems to make the most sense to keep the clients functioning. Using threads from python libraries allows for easy implementation of this process and allows for me to give up control of the threads and the 'lock' which is the way for the processor to know which process is using valuable resources like a file or a directory that if 2 clients tried to access would cause issues. The lock allows for processes to share these types of material without causing issues such as dead lock.

re : Re is a library that I used to help me find the special characters in my passwords. I used this because it allows for looping through the string and finding the characters that match the ones I was asking it to look for, returning a boolean, True or False statement when found allowing for simpler coding for situations like password checking.

maskpass: This is a library that I implemented on my client so that my passwords were hidden from view when they were being entered. This worked by "masking" any thing the client typed into the input prompt in python with (***) so that they could not be seen but still saved the password as a plain text. This is a type of keylogging library that follows all the keyboard inputs so as to know when to "enter" and when to (*) an input.

bcrypt : Bcrypt is a library in python that allows for hashing, which is what I used it for. It takes in an input or in this case password and a salt and hashes the password for me. It also creates a salt for me which is a randomly generated value or phrase that is placed in front of the password and then hashed so that if 2 passwords are the same in plaintext they will never be hashed the same. This is always for securing the user passwords and security questions in my code.

Citations

Socket connection outside network :

<https://docs.python.org/3/library/socket.html>

C Code or Bash Script for server configurations:

<https://greenwebpage.com/community/how-to-kill-a-process-using-kill-command-in-linux/>

<https://www.linux-magazine.com/Online/Features/OpenSSL-with-Bash>

<https://www.cyberciti.biz/faq/unix-linux-bsd-appleosx-bash-assign-variable-command-output/>

https://www.w3schools.com/bash/bash_cd.php

<https://www.geeksforgeeks.org/linux-unix/shell-scripting-shell-variables/>

<https://www.digitalocean.com/community/tutorials/execute-command-shell-script>

<https://www.geeksforgeeks.org/linux-unix/bash-scripting-introduction-to-bash-and-bash-scripting/>

<https://www.geeksforgeeks.org/linux-unix/how-to-run-bash-script-in-linux/>

Multiple client connections simultaneously:

<https://www.geeksforgeeks.org/python/socket-programming-multi-threading-python/>

<https://docs.python.org/3/library/threading.html>

<https://www.geeksforgeeks.org/python/handling-a-threads-exception-in-the-caller-thread-in-python/>

https://www.youtube.com/watch?v=5G_bNVKdECkhttps://www.youtube.com/watch?v=5G_bNVKdECk

<https://www.geeksforgeeks.org/python/how-to-create-a-new-thread-in-python/>

<https://www.stratascratch.com/blog/python-threading-like-a-pro/>

<https://www.geeksforgeeks.org/python/multithreading-python-set-1/>

Certificates for Server:

https://docs.python.org/3/library/ssl.html#ssl.create_default_context

<https://pythontic.com/ssl/sslsocket/introduction>

<https://bugs.python.org/issue27391>

<https://www.geeksforgeeks.org/linux-unix/practical-uses-of-openssl-command-in-linux/>

<https://medium.com/@LukV/mutual-tls-mtls-a-deep-dive-into-secure-client-server-communication-bbb83f463292>

<https://blog.devolutions.net/2020/07/tutorial-how-to-generate-secure-self-signed-server-and-client-certificates-with-openssl/>

<https://www.jscape.com/blog/how-to-generate-a-client-certificate>

<https://www.geeksforgeeks.org/python/python-subprocess-module/>

Mysql database on python:

<https://wiki.cs.jmu.edu/student/python/virtualenv>

<https://docs.python.org/3/library/venv.html>

<https://www.geeksforgeeks.org/python/python-sqlite-cursor-object/>
<https://www.geeksforgeeks.org/python/enumerate-in-python/>
<https://www.geeksforgeeks.org/python/python-dictionary/>
<https://www.geeksforgeeks.org/python/python-database-tutorial/>
https://www.geeksforgeeks.org/python/python-mongodb-find_one_and_update-query/
<https://www.geeksforgeeks.org/python/mongodb-and-python/>
<https://www.geeksforgeeks.org/sql/foreign-key-constraint-in-sql/>
<https://www.geeksforgeeks.org/python/inserting-variables-to-database-table-using-python/>
<https://www.geeksforgeeks.org/python/introduction-to-sqlite-in-python/>
https://www.w3schools.com/sql/sql_foreignkey.asp
<https://www.geeksforgeeks.org/python/check-if-string-contains-substring-in-python/>
<https://docs.python.org/3/library/sqlite3.html#sqlite3.Connection.commit>
<https://www.geeksforgeeks.org/python/python-sqlite-update-data/>
<https://www.geeksforgeeks.org/python/python-nested-dictionary/>
<https://www.geeksforgeeks.org/python/python-accessing-key-value-in-dictionary/>
<https://www.tutorialspoint.com/python-get-particular-nested-level-items-from-dictionary>
https://www.tutorialspoint.com/python/dictionary_len.htm

RSA for password encryption:

<https://docs.python.org/3/library/hashlib.html>
<https://www.geeksforgeeks.org/python/how-to-hash-passwords-in-python/>
<https://www.geeksforgeeks.org/python/hiding-and-encrypting-passwords-in-python/>
<https://www.geeksforgeeks.org/python/hashing-passwords-in-python-with-bcrypt/>
checks password for criteria:
<https://www.geeksforgeeks.org/python/python-test-if-string-contains-any-uppercase-character/>
<https://www.geeksforgeeks.org/dsa/python-program-check-string-contains-special-character/>
<https://www.geeksforgeeks.org/python/python-any-function/>
lockout method for if suspicious behavior: <https://realpython.com/python-gil/>
Server and client as an object
<https://www.geeksforgeeks.org/python/create-a-python-subclass/>
<https://www.geeksforgeeks.org/python/python-object/>

Handling directories and Tuple:

<https://www.geeksforgeeks.org/python/delete-a-directory-or-file-using-python/>
<https://docs.python.org/3/library/stdtypes.html#tuple>
<https://www.geeksforgeeks.org/python/Iterate-over-a-tuple-in-Python/>
<https://www.geeksforgeeks.org/python/python-get-dictionary-keys-as-a-list/>

Fun stuff:

<https://www.asciiart.eu/text-to-ascii-art>

<https://www.asciiart.eu/text-to-ascii-art>

<https://www.geeksforgeeks.org/python/python-raw-strings/>

<https://www.geeksforgeeks.org/python/clear-screen-python/>

```
mkdir ~/project
virtualenv -p /usr/bin/python3 ~/project
cd ~/project
source bin/activate
pip install --upgrade pip
pip install --upgrade setuptools
pip install bcrypt
pip install maskpass
```

Commands I used to create certificate:

```
openssl ecparam -name prime256v1 -genkey -noout -out ca.key
openssl req -new -x509 -sha256 -key ca.key -out ca.crt
```

```
openssl ecparam -name prime256v1 -genkey -noout -out server.key
openssl req -new -sha256 -key server.key -out server.csr
openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out server.pem -days
1000 -sha256 -extfile server-extensions.txt
```

Client commands:

```
openssl req -new -sha256 -key client.key -out client.csr
openssl ecparam -name prime256v1 -genkey -noout -out client.key
openssl x509 -req -in client.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out client.pem -days
1000 -sha256 -extfile client-extensions.txt
```

Certificate Information: Server

Country Name: US

State or Province: VA

Locality Name: Harrisonburg

Organization Name: KatherineServerLLC

Organizational Unit: Security

Common Name: Server

Email: your email

challenge Password: cookie

company name: Blank

Certificate Information: Client

Country Name: US

State or Province: VA

Locality Name: Harrisonburg

Organization Name: KatherineServerLLC

Organizational Unit: Security

Common Name: Client

Email: your email

challenge Password: cookie

company name: Blank